

Ejercicio 1 — Biblioteca

Queremos desarrollar un sistema para representar los materiales que posee una biblioteca. La biblioteca puede tener **libros** y **revistas**.

- Todo **material** se caracteriza por:
 - título
 - año de publicación
 - precio
- Los **libros** son materiales que además tienen:
 - autor
 - cantidad de páginas
- Las **revistas** son materiales que además tienen:
 - número de edición
 - cantidad de artículos

Se pide:

A) Implemente la jerarquía de clases correspondiente. Decida qué clase debe ser la superclase y cuáles sus subclases. Declare los atributos, constructores, getters y setters necesarios.

B) Todo material debe poder responder al mensaje `calcularValor()`.

- Para un libro, el valor se calcula como el precio más un adicional de \$100 por cada 100 páginas.
- Para una revista, el valor se calcula como el precio más \$500 si tiene más de 10 artículos.

Piense **en qué clase/s debe declarar este método** y cuáles deberían implementarlo.

C) Todo material debe responder al mensaje `toString()`, que debe devolver un String con toda su información y el valor calculado mediante `calcularValor()`.

D) Suponga ahora que todos los materiales tienen un método `calcularDescuento()` que inicialmente devuelve un descuento del **5% del precio**.

Para los libros, el descuento debe ser del **10%** si tienen más de 300 páginas.

Para las revistas, el descuento debe ser del **15%** si tienen más de 20 artículos.

Piense dónde conviene definir `calcularDescuento()` y dónde redefinirlo.

E) Al redefinir los métodos necesarios en las subclases, utilice `super` cuando corresponda para reutilizar el comportamiento de la superclase.

Ejercicio 2 — Personas de una institución

Una institución necesita representar a las personas que forman parte de ella.

Una **Persona** tiene:

- nombre
- DNI
- edad

Existen dos tipos:

- **Alumno**, que además tiene carrera y año que está cursando.
- **Profesor**, que además tiene materia que dicta.

A)

Implemente la jerarquía de clases, con constructores, getters y setters.

B)

Tanto **Persona** como sus subclases deben responder a `toString()`.

La representación de una persona debe ser:

```
"Nombre: Juan, DNI: 12345678, Edad: 20"
```

La de un alumno debe agregar:

```
"Carrera: Informática, Año: 2"
```

La de un profesor debe agregar:

```
"Materia: Programación"
```

Piense dónde conviene implementar cada `toString()` y cómo utilizar `super`.

C)

Agregue a **Persona** un método `esMayorDeEdad()` que indique si la persona tiene 18 años o más.

Pruebe el método con un alumno y un profesor.

D)

Realice un programa principal que cree una persona, un alumno y un profesor y muestre la representación de cada uno.

Ejercicio 3 — Cuenta bancaria

Queremos representar cuentas bancarias.

Una **Cuenta** tiene:

- número de cuenta
- titular
- saldo

Existen dos tipos:

- **Caja de ahorro**
- **Cuenta corriente**

A)

Implemente la jerarquía correspondiente.

B)

Toda cuenta debe responder a:

depositar(monto)

El depósito se realiza de la misma manera para cualquier cuenta.

C)

Toda cuenta debe responder a:

retirar(monto)

Pero cada tipo de cuenta tiene reglas diferentes:

- En una **caja de ahorro**, solamente se puede retirar si hay saldo suficiente.
- En una **cuenta corriente**, se puede retirar hasta \$50.000 por debajo del saldo disponible.

Piense qué método debería estar en la superclase y cómo debería resolverse la diferencia entre las subclases.

D)

Toda cuenta debe responder a `toString()`, mostrando número, titular y saldo.

E)

Agregue un método `calcularInteres()`:

- La caja de ahorro obtiene un interés del 3% sobre su saldo.
- La cuenta corriente obtiene un interés del 1%.

Realice un programa que cree una cuenta de cada tipo, haga algunos depósitos y retiros y muestre sus datos.

Ejercicio 4 — Competencia de atletismo

Una organización quiere desarrollar un sistema para registrar los resultados de una competencia de atletismo.

La competencia tiene **N participantes**. De cada participante interesa conocer:

- nombre
- DNI
- edad

Además, existen dos tipos de participantes:

- **Corredores**, que registran sus tiempos en distintas carreras.
- **Lanzadores**, que registran sus distancias obtenidas en distintos lanzamientos.

Cada participante puede tener hasta **10 resultados** registrados.

A) Jerarquía

Implemente las clases necesarias para representar esta situación.

Piense:

- ¿Cuál debería ser la superclase?
- ¿Debería ser abstracta?
- ¿Qué atributos son comunes?
- ¿Qué comportamiento es diferente entre corredores y lanzadores?

La superclase debe tener un método:

`calcularResultado()`

que permita obtener un resultado representativo del participante.

B) Registrar resultados

Cada participante debe permitir registrar un nuevo resultado.

Para un **corredor**, el resultado representa un tiempo en segundos.

Para un **lanzador**, el resultado representa una distancia en metros.

Los resultados deben almacenarse en un **vector**.

C) Resultado representativo

El comportamiento de `calcularResultado()` será diferente:

- Para un **corredor**, debe devolver el **menor tiempo** registrado.
- Para un **lanzador**, debe devolver la **mayor distancia** registrada.

Piense qué método debería ser abstracto y en qué clase debería declararse.

D) Clasificación

Todo participante debe responder a:

`obtenerClasificacion()`

- Para un corredor:
 - devuelve **"Destacado"** si su mejor tiempo es menor a 12 segundos.
 - devuelve **"Regular"** en caso contrario.
- Para un lanzador:
 - devuelve **"Destacado"** si su mejor distancia es mayor a 70 metros.
 - devuelve **"Regular"** en caso contrario.

E) Representación

Todo participante debe responder a `toString()`.

Debe mostrar:

- nombre
- DNI
- edad
- resultados registrados
- resultado representativo
- clasificación

Cada subclase debe agregar además la información que considere propia de su tipo.

Utilice `super` cuando corresponda.

F) Programa principal

Realice un programa que:

1. Cree un corredor y un lanzador.
2. Registre varios resultados para cada uno.
3. Muestre la representación `String` de ambos.

4. Muestre cuál es el resultado representativo de cada participante.